Jev-Backed Function Calling: Iterative Prompting with Verification Feedback

Anonymous Author(s)

Abstract

Function-calling models must choose a function and input values that match the user's request. Correct format alone does not ensure correct choices. We present Jev-Backed Function Calling (**JBFC**), an inference-time method that uses TypeSafe's System One (Decision) model *Jev* to check an LLM's proposed call. *Jev* answers two binary questions: whether the function is appropriate and whether its arguments are correct. If either check fails, we add the proposal and judgments to a growing history placed before the original user request. The LLM uses this feedback to revise both the function and arguments. Acceptance requires both checks and the schema check to pass on the same proposal. A repair limit is chosen on development data and fixed before testing. Our code compares six methods on 2,748 test examples from a 3,051-example Berkeley Function Calling Leaderboard (BFCL) subset, measuring call correctness, function choice, argument correctness, time, and cost. Experiments have not yet been run. They will test whether accumulated verification feedback improves calls and whether repairs justify their added cost.

1 Introduction

To carry out a user's request correctly, a function-calling system must choose the right function and provide the right argument values. A language model, called the generator, makes these choices. However, a call can have the required format and still use the wrong function or values. For example, a travel assistant may choose the wrong booking function or swap the departure and destination. A JSON schema check verifies required inputs and their types, but cannot guarantee that the call matches the user's request.

Tool-use training and evaluation have improved function-calling systems [2, 3, 1]. Even so, the generated function and arguments can still be wrong. We study whether checks from a separate decision model can guide the LLM to improve its next proposal before execution.

JBFC uses *Jev*, TypeSafe's System One model, to answer binary questions about an LLM's function choice and arguments [12]. When a check fails, we keep the proposed call and its judgments and place all earlier records before the original user request. The LLM then proposes a new call using this accumulated feedback. It can change both the function and argument values. **JBFC** accepts a proposal only when both binary checks and the schema check pass on that proposal.

Earlier work already uses checks and feedback to fix calls. ToolVerifier checks tool choice and parameters [9]; ToolCritic gives feedback on tool-use errors [10]. ReAct uses reasoning, actions, and observations to guide later steps [14]. **JBFC** also carries feedback forward in the prompt, but its feedback comes from checking proposed calls before execution. Our contribution is this specific use of two *Jev* judgments and accumulated prompts, together with an evaluation of its effects and costs.

We compare **JBFC** with direct calling, schema repair, self-revision, and single-check versions. A repair limit chosen on development data controls the comparison. We measure errors fixed or introduced by repair, and keep call correctness separate from acceptance, time, and cost.

2 Related Work

Tool use and benchmarks. Gorilla studies API-call generation [2]; ToolLLM studies tool-use training and evaluation [3]. BFCL tests function calling across many settings [1]. We use its single-turn Python categories to study decisions made before a function runs. API-Bank offers dialogues with executable API calls [4]. StableToolBench uses caching and simulation to handle changing APIs [5]. These suggest broader tests beyond our BFCL subset.

Checks and feedback. Self-Refine uses self-generated feedback [6], Reflexion uses written feedback [7], and CRITIC uses feedback from tools [8]. Self-correction can turn correct answers into wrong ones [11], so we measure both fixes and new errors. ToolVerifier compares alternatives to check tool choice and parameters separately [9]. ToolCritic identifies tool-use errors and returns feedback [10]. **JBFC** checks complete proposals with a fixed external model through an API and limits repairs.

Accumulated prompts. ReAct combines reasoning with actions and observations from an environment [14]. Our method uses a similar idea of carrying earlier feedback into the next prompt. Each record contains a proposed function call and verification results, rather than an execution result. The LLM receives all records before the unchanged user request. We do not require a generated reasoning trace or run functions during repair.

***Jev* as a judge.** Li et al.'s *JEV-as-a-Judge* [13] checks finished outputs with probabilities for defined verdicts. If a verdict is uncertain, an independent, stronger judge decides. With fixed rules for when to use the stronger judge, it lowers

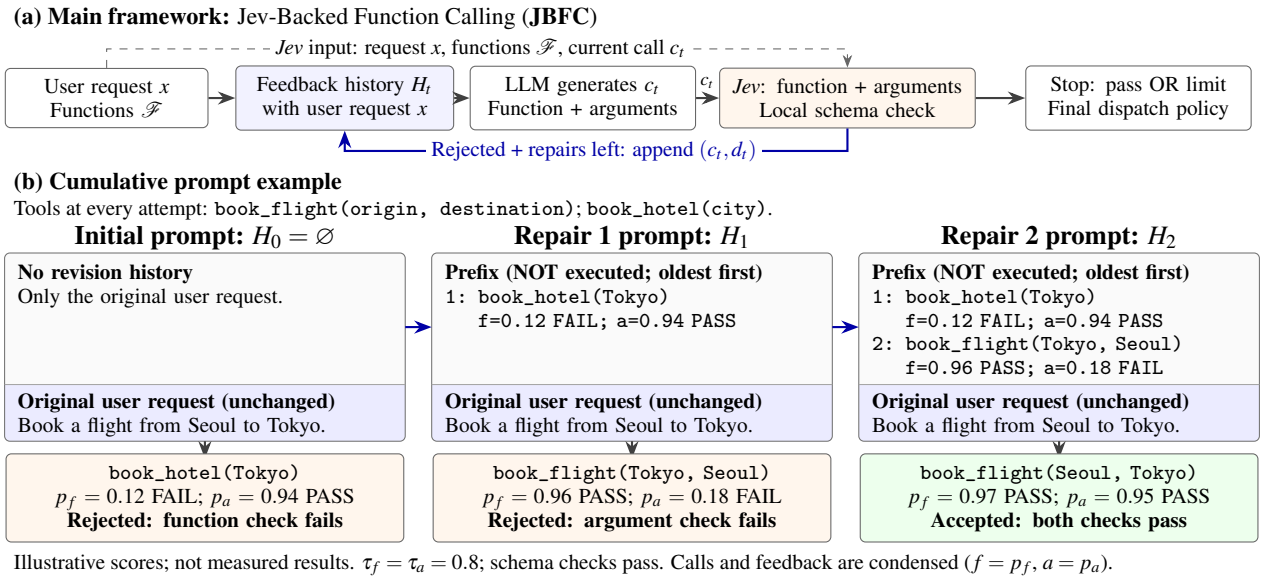


Figure 1: **JBFC** and cumulative prompt feedback. Top: the LLM reads earlier calls and judgments (H_t), then the original request (x), and generates a call (c_t) with a function and arguments. Failed checks add a record for another attempt. Bottom: growing history precedes the unchanged request; new calls appear below. Argument checks concern the proposed function. No function runs during repair.

judging cost while keeping similar accuracy on some tasks. It has limits on difficult reasoning, misleading answer styles, and writing judged without reference information. The study does not test tool or action selection inside agents. **JBFC** checks proposed function calls and returns failed checks to the same generator for revision. We test the specific design of **JBFC**.

3 Method

3.1 Propose and Check a Call

Let x be the user request, $\mathscr{F}$ the available functions, and G the generating LLM. A proposal is $c_t = (f_t, a_t)$, with function f_t and arguments a_t, or $c_t = \varnothing$ for no call. We study single-call-or-no-call decisions.[1] Local code checks readability, function availability, required arguments, and types. Write $S(c_t) = 1$ when these checks pass.

Binary verification: JBFC. For each readable proposal, one *Jev* request checks the function choice and, when a call is present, its arguments:

$$c_t = G(H_t, x, \mathscr{F}), \tag{1}$$

$$p_f^{(t)} = J_f(x, \mathscr{F}, c_t), \qquad p_a^{(t)} = J_a(x, \mathscr{F}, c_t). \tag{2}$$

Here H_t contains earlier proposals and feedback and is initially empty. The function `Noul` question asks whether the function choice, including no call, fits the request. The argument question checks required values, their support in the request, and their parameter roles. For no call, it is skipped and $p_a^{(t)}$ is undefined. A selected function with missing arguments still receives both checks. Both must pass for a call.

[1] We treat single-turn decisions as the basic unit of function calling. Parallel calling combines multiple such decisions, while multi-turn interactions repeat them with updated context. We therefore evaluate the verification and repair loop on this basic unit and exclude parallel and multi-turn settings from this study.

3.2 Accept or Repair a Call

The acceptance rule is

$$A_t = \begin{cases} S(c_t)\,\mathbf{1}[p_f^{(t)} \geq \tau_f], & c_t = \varnothing, \\ S(c_t)\,\mathbf{1}[p_f^{(t)} \geq \tau_f]\,\mathbf{1}[p_a^{(t)} \geq \tau_a], & \text{otherwise.} \end{cases} \tag{3}$$

We start with $\tau_f = \tau_a = 0.8$. These are untuned defaults. Noul returns a probability of yes; the winning-verdict confidence in JEV-as-a-Judge is a different quantity [13]. A confident no rejects a proposed call.

The repair limit r is any nonnegative integer, chosen on development data. It permits at most $r+1$ LLM proposals, with early stopping after acceptance. Every new proposal receives all applicable checks; an earlier pass is never reused. Reference answers and offline scores stay out of model requests. API failures stop the affected request after limited transport retries, which are separate from repairs.

Feedback before the user request. If $A_t = 0$ and another repair is allowed, d_t contains returned probabilities, pass/fail flags, thresholds, and local errors. For no call, the argument check is marked as skipped, with no verdict or probability. We append the record:

$$H_{t+1} = H_t \,\|\, [(c_t, d_t)]. \tag{4}$$

Here $\|$ appends a record. The next prompt places all records, from oldest to newest, before the original request. Each has a round number, proposed calls, and feedback. Original messages keep their content and order, and all schemas remain available. Feedback about a wrong function can lead to a different function next.

The prefix marks proposals as not executed and asks the LLM to reconsider function choice and arguments. Records are JSON in an ordinary user message; system instructions and tool schemas remain separate. *Jev* returns probabilities, so we do not invent explanations. Unreadable proposals receive local parsing feedback without a *Jev* request. *Jev* sees only the original request, schemas, and current proposal, keeping its check focused on that proposal.

3.3 Return and Record the Result

The main `execute_last` policy returns the final proposal for scoring even after a failed check. The optional `abstain` policy withholds rejected calls; we still score them offline. An application callback can run only at the final dispatch boundary, never during repair. The benchmark never executes functions, so correctness does not establish successful real-world execution.

Records contain proposals, checks, feedback, stopping reason, returned model versions, usage, and timings. API keys stay outside records. The manifest records data, source-code hashes, models, routes, and runtime settings. All six methods share one initial LLM proposal per example and repeated run. Each method is charged for all work it would use alone, including the shared first proposal and tokens used for accumulated history.

4 Experiments

4.1 Data and Evaluation

We prepare 3,051 BFCL examples from seven categories at repository revision `6ea57973c7a6`[2]. Table 1 shows downloaded and test counts. We sort each category by the SHA-256 hash of `seed:id`. The first $\max(1, \lfloor 0.1 N_k \rfloor)$ examples form the development set; N_k is the category size. Seed 1729 gives 303 development and 2,748 test examples. Model experiments have not been run.

For categories with reference calls, the original Python abstract syntax tree (AST) checker tests function names and arguments against allowed answers. Irrelevance categories require no call. Live relevance checks only call presence, not function or argument correctness. We exclude it from main accuracy and verifier calibration that needs those labels. The main score covers 2,733 test examples per repeated run. We report category accuracy and a micro-average, which weights examples equally. Category scores show what the larger live categories may hide in the average.

[2]Full revision: `6ea57973c7a6097fd7c5915698c54c17c5b1b6c8`. The saved setup file lists source URLs, file hashes, format changes, and example IDs.

Category	All	Test	Direct	**JBFC**
Simple Python	400	360	–	–
Multiple	200	180	–	–
Irrelevance	240	216	–	–
Live simple	258	233	–	–
Live multiple	1053	948	–	–
Live irrelevance	884	796	–	–
Live relevance[†]	16	15	–	–
Total	3051	2748	–	–

Table 1: BFCL example counts and test accuracy (%). Counts describe the prepared data. [†]Live relevance checks only whether a call is present; total accuracy excludes these 15 cases. – means the experiment has not been run.

BFCL's "multiple" categories offer several functions to choose from, without parallel calls. We exclude parallel calls, multi-turn state, web search, memory, and live service execution. "Live" names a category; the experiment makes no network request to execute a function. The official AST check compares call structure with annotated alternatives. Passing it does not prove successful execution, authorization, or completion of a larger user task.

4.2 Models and Methods

The generator is `gpt-5.4-mini-2026-03-17`, with reasoning effort `none`; *Jev* is pinned to `jev-1.13.0`. We use OpenAI Responses and System One through an APIM gateway, preserve supplied schemas, and plan three repeated runs with fixed prompts. Models, thresholds, and prompts can be changed, but their selection must use development data before the test setup is frozen.

We compare six methods. *Direct* uses the initial LLM proposal. *Schema-only* repairs code-detected errors. *Self-refine* uses a fixed self-review instruction, a generic control rather than the full Self-Refine method [6]. **JBFC-F** and **JBFC-A** obtain the applicable Noul judgments but use only the function or argument check, respectively, with schema checks. Feedback exposes only that check. With no argument check, **JBFC-A** accepts no call on schema checks alone. **JBFC** requires all applicable checks. Repair methods keep earlier proposals and feedback before the original request and share a maximum proposal count. Actual time and cost can differ.

Choosing the repair limit. We choose r before testing. The initial candidates are $\mathscr{R} = \{0,1,2,3,4,5\}$, a configurable search range. **JBFC** has no built-in limit of five. In three development runs, we allow up to $\max \mathscr{R}$ repairs and stop early when all applicable checks pass. Neither model sees the limit, and no function runs during repair. Each smaller limit can therefore be tested on the same saved sequence without more API calls. *Jev* thresholds stay at 0.8 during this search.

We choose the limit with the highest final-call accuracy on development data. Ties choose the smaller limit:

$$r^* = \min \arg\max_{r \in \mathscr{R}} \widehat{\mathrm{Acc}}_{\mathrm{dev}}(r). \tag{5}$$

Method	Call	Func.	Args.	Sec.	USD/1k
Direct	–	–	–	–	–
Schema repair	–	–	–	–	–
Self-refine	–	–	–	–	–
JBFC-F	–	–	–	–	–
JBFC-A	–	–	–	–	–
JBFC	–	–	–	–	–

Table 2: Methods on the BFCL test set. The repair limit is chosen on development data. Call and function accuracy (%) exclude live relevance. Argument accuracy (%) uses calls with the correct function and argument labels. – means unavailable or unrun.

r	Dev Acc. (%)	Repairs	Sec.	USD/1k
0	–	–	–	–
1	–	–	–	–
2	–	–	–	–
3	–	–	–	–
4	–	–	–	–
5	–	–	–	–

Table 3: Development data choose the repair limit for **JBFC**. We choose the highest call accuracy; ties choose the smaller limit. Accuracy excludes live relevance. – means no result is available or the experiment has not been run.

The score covers 302 development examples per run, excluding the one live-relevance example. We save the chosen limit, settings, and development-record hashes before three test runs compare all six methods. Test scores never choose the limit. Table 3 shows the development search for **JBFC**. Optional smaller-limit and `abstain` analyses use only saved **JBFC** sequences. Cost and time cover only the selected prefix. With $r = 0$, the initial proposal is checked but never revised.

4.3 Measures, Cost, and Uncertainty

We report final call accuracy, function accuracy, and argument accuracy conditional on the correct function. Every score includes its denominator. Live relevance is excluded from these accuracy measures and reported separately. We compare initial and final candidates to identify function changes, argument fixes, and new errors after feedback.

Acceptance is distinct from correctness. Coverage is the fraction accepted; accepted precision is accuracy among accepted calls. Selective risk is one minus this precision. False acceptance conditions on incorrect calls, and false rejection on correct calls. We also record corrections, regressions, repairs, API errors, and Brier scores where matching probability and correctness labels exist. These measures show whether feedback corrects rejected errors or instead introduces new ones.

Reported costs count all recorded generation and verification work, with shared initial calls charged to each method. Growing history adds input tokens on later attempts, and these tokens count toward cost. We report mean and p95 time, token use, estimated cost, and per-run scores. Test costs exclude the development search. Missing usage or prices remain unavailable, not zero. Charges for failed requests may be missing. Default prices on October 15, 2026, per million tokens are \$0.75/\$0.075/\$4.50 for uncached/cached/output LLM tokens and \$0.042 for *Jev* input, with free *Jev* output; gateway fees are excluded.

We compare each repair method with Direct on matched example IDs, repeated runs, and shared initial proposals. We average each example's accuracy differences across runs, then bootstrap example IDs 2,000 times with seed 1729. The middle 95% of values gives the interval. Tables 2–3 are unrun templates. Partial or synthetic runs cannot fill them.

4.4 Study Limits

The verifier can accept a wrong call or share the generator's mistake. Repair can introduce errors, and later proposals may differ from development examples. Accumulated feedback can reinforce an earlier mistake or distract the LLM from the original request. The history also grows with each repair, increasing context use and cost. Binary verdicts do not explain how to fix an error. Higher acceptance alone does not show improvement.

The study uses one model pair, mostly English requests, and single-turn calls. Public examples may overlap with training data. We keep four documented annotation inconsistencies in the main score. The candidate grid and small development set can affect the chosen repair limit. A common limit chosen for **JBFC** may disadvantage other methods. Further studies should tune each method separately and test more models and domains. API failures and incomplete runs remain visible. Conclusions must follow measured results, including no gain or worse performance.

5 Conclusion

JBFC uses *Jev*'s function and argument judgments as accumulated prompt feedback. After a failed check, earlier proposals and judgments appear before the original user request, allowing the LLM to revise the complete call. A development-chosen repair limit bounds this process. The code separates correctness, acceptance, and cost. Experiments remain to be run; they will test whether this feedback improves function calling and whether the gains justify the added work.

Acknowledgements

This work was created as part of the No-More-Slops group activities at Pseudo Lab.

References

[1] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *International Conference on Machine Learning*, PMLR 267, pp. 48371–48392, 2025. https://proceedings.mlr.press/v267/patil25a.html.

[2] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems*, 37:126544–126565, 2024. https://arxiv.org/abs/2305.15334.

[3] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2307.16789.

[4] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Empirical Methods in Natural Language Processing*, pp. 3102–3116, 2023. https://aclanthology.org/2023.emnlp-main.187/.

[5] Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. StableToolBench: Towards Stable Large-Scale Benchmarking on Tool Learning of Large Language Models. In *Findings of ACL*, pp. 11143–11156, 2024. https://aclanthology.org/2024.findings-acl.664/.

[6] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023. https://arxiv.org/abs/2303.17651.

[7] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023. https://arxiv.org/abs/2303.11366.

[8] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2305.11738.

[9] Dheeraj Mekala, Jason Weston, Jack Lanchantin, Roberta Raileanu, Maria Lomeli, Jingbo Shang, and Jane Dwivedi-Yu. ToolVerifier: Generalization to New Tools via Self-Verification. *arXiv:2402.14158*, 2024. https://arxiv.org/abs/2402.14158.

[10] Hassan Hamad, Yingru Xu, Liang Zhao, Wenbo Yan, and Narendra Gyanchandani. ToolCritic: Detecting and Correcting Tool-Use Errors in Dialogue Systems. *arXiv:2510.17052*, 2025. https://arxiv.org/abs/2510.17052.

[11] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large Language Models Cannot Self-Correct Reasoning Yet. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2310.01798.

[12] TypeSafe. Models: *Jev* and the System One API. Technical documentation, 2026. Accessed October 15, 2026. https://docs.typesafe.ai/models.

[13] Yubo Li, Yidi Miao, Ramayya Krishnan, and Rema Padman. JEV-as-a-Judge: Accept When Confident, Escalate When Unsure. *arXiv:2609.26550*, version 5, October 9, 2026. https://arxiv.org/abs/2609.26550v5.

[14] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023. https://arxiv.org/abs/2210.03629.